Deliverable 2 - INF201

G6_1: Zuzanna Magdalena Smolinska and Sondre Aasheim

Submission deadline: 30th Oct 2026

Hand-in information:

- This is a group submission, so one person submits on behalf of the pair/group
- Submit the Jupyter notebook (ipynb) and a pdf of the same notebooks
  - Should contain code, results/plots and discussion in markdown cells
- Also submit the output files generated by your code
- Include a link to your GitHub repository and a log of the commits
  - To generate the log, paste this into the terminal after your last commit: `git log --graph --pretty=format:"%h - %an, %ar : %s" > git_log.txt`
  - The repository should contain a `requirements.txt` with information on your virtual environment
  - Note: Remember to create a new virtual environment for each deliverable
- Include a discussion of GenAI use, for example a summary of the chat log as you have done earlier
  - This can be a separate file or be included in the Jupyter notebook
- Make a short (5 mins) video discussing your code and results
  - Note that you both have to contribute to the video
  - Either by recording it together
  - Or recording parts separately and editing it together

Scenario - Campus Coffee Shop Sales Pipeline

You have been hired as a Python developer for the Campus Coffee Shop Chain. The chain operates several coffee stands across different campus buildings. This is an international campus, so the stands accept payment in several currencies.

Each cash register saves transaction logs as text files. However, the legacy system has several issues:

- Mixed Character Encodings: Older registers write text files using ISO-8859-1 (Latin-1) encoding, while newer registers write files using UTF-8.
- System Noise: Registers intersperse valid sales lines with printer and network debug log lines.
- Nested Folder Hierarchy: Log files are stored across nested subdirectories organized by year, store, and register ID.
- Data Errors: System glitches record occasional extreme price outliers (e.g., $999.00), and prices in NOK are by error registered using EUR-scale values (e.g., 4.50 NOK instead of 45.00 NOK).

Your task is to set up Python code that discovers all log files, cleans the transaction data, calculates price metrics and generates summary reports. Note: In this assignment, you write the functions you will use in tasks 1-4, and then put everything together in task 5. You should therefore read through the entire assignment text before you start, to fully understand what you will use each function for.

Overview of inputs and outputs for this assignment

Inputs (in `data/` folder)

- You generate the input data on you own laptop by running the script `generate_data_for_d2.py`
- `data/store_config.json`: A nested JSON file containing store metadata.
- `data/2026/...`: A folder hierarchy containing subdirectories with .log files in mixed character encodings.

Outputs (files your code should generate)

- `cleaned_transactions.csv`: A CSV table containing all parsed, outlier-cleaned, and currency-corrected transactions.
- `hourly_revenue.json`: A JSON report containing total revenue per hour per item.
- `status_pivot_table.csv`: A CSV file containing a pivot table of total transaction revenue grouped by item across status categories.
- `item_refund_ratios.csv`: A CSV table containing refund ratios for each item type.

0.1: Imports and paths

```
import json
import re
from pathlib import Path

import datetime
import numpy as np
import pandas as pd

DATA_DIR = Path("data")              # location of the input files
OUTPUT_DIR = Path("output")          # location where our generated
reports will be saved

OUTPUT_DIR.mkdir(exist_ok=True)      # no error if our generated folder
already exists
```

0.2: Environment check

```
"""
If this fails, the notebook isn't using the .venv kernel,
or the "data" folder hasn't been generated yet.
"""

print("numpy version: ", np.__version__)
```

```
print("pandas version:", pd.__version__)
print("data/ folder found:", DATA_DIR.exists())
```

```
numpy version:  2.5.4
pandas version: 3.1.0
data/ folder found: True
```

Task 1: File ingestion and regular expressions

1. Write a robust file reader function:
   - Use `pathlib.Path` to handle file paths.
   - The function should be able to handle both `UTF-8` and `ISO-8859-1/Latin-1` encoding. You may assume that these are the only two encodings the files can have. Note that the function should handle the encoding, you may not hard-code which encoding each file has.
   - Return a tuple containing two strings: `(file_text_content, encoding_used)`.
2. Write a log parser function that uses regular expressions. Log files contain valid sales lines mixed with system noise. Valid line format: `[2026-10-15 08:30:00] [POS_01] item=Latte price=4.50 EUR (SUCCESS)` Noise lines to ignore: `--- PRINTER PAPER LOW WARNING ---` `=== NETWORK SYNC OK ===`
   - Construct a regular expression (`re` module) to capture the 6 key fields from valid lines:
     - `timestamp` (string, format: `YYYY-MM-DD HH:MM:SS`)
     - `register_id` (string, e.g., `POS_01`)
     - `item` (string, e.g., `Latte`)
     - `price` (float, e.g., `4.50`)
     - `currency` (string, e.g., `EUR`, `NOK`, `USD`)
     - `status` (string: `SUCCESS`, `REFUND`, or `CANCELLED`)
   - Return a list of Python dictionaries, where each dictionary represents one valid transaction record.

1.1: Robust file reader

```
def read_log_file(path: str | Path) -> tuple[str, str]:     # the function returns a tuple holding two strings
    """
    Reading a text file encoded as either UTF-8 or ISO-8859-1 (Latin-1).

    We try UTF-8 first, then fall back to Latin-1 if decoding fails.
    UTF-8 must come first as Latin-1 can decode any bytes and will always succeed.
    """

    path = Path(path)

    try:
```

```
        text = path.read_text(encoding="utf-8")         # will fail on
invalid bytes
        return text, "utf-8"
    except UnicodeDecodeError:
        text = path.read_text(encoding="iso-8859-1")    # will never
fail
        return text, "iso-8859-1"
```

1.1.1: Code verification

```
"""
Write the word *café* in both encodings and check that each file is
read back correctly.
The letter *é* is stored as 2 bytes in UTF-8 but 1 byte in Latin-1.
"""

test_dir = Path("tmp_test")
test_dir.mkdir(exist_ok=True)

(test_dir / "utf8.log").write_text("café", encoding="utf-8")
(test_dir / "latin1.log").write_text("café", encoding="iso-8859-1")

print(read_log_file(test_dir / "utf8.log"))
print(read_log_file(test_dir / "latin1.log"))
```

```
('café', 'utf-8')
('café', 'iso-8859-1')
```

1.2.1: Log parser with regular expressions

```
LOG_PATTERN = re.compile(
    r"^\[(?P<timestamp>\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2})\]"   # eg
[2026-10-15 08:30:00]
    r" \[(?P<register_id>[A-Z]+_\d+)\]"                          # eg
[POS_01]
    r" item=(?P<item>\w+)"                                       # eg
item=Latte
    r" price=(?P<price>\d+(?:\.\d+)?)"                           # eg
price=4.50
    r" (?P<currency>[A-Z]{3})"                                   # eg
EUR / NOK / USD
    r" \((?P<status>SUCCESS|REFUND|CANCELLED)\)$"                # eg
(SUCCESS)
)


def parse_log(text: str) -> list[dict]:     # the function returns a
list of dictionaries
    """
    Parse a log file and return a list of dictionaries, one for each
```

```
valid line.
    Invalid lines are ignored.
    """

    records = []

    for line in text.splitlines():
        match = LOG_PATTERN.match(line.strip())  # removes stray
spaces, line endings
        if match is None:                        # if not a sales
line, skip it
            continue

        record = match.groupdict()                # named groups ->
dict
        record["price"] = float(record["price"])  # captured as text,
convert to float
        records.append(record)

    return records
```

1.2.2: Code verification

```
sample_text = """
[2026-10-15 08:30:00] [POS_01] item=Latte price=4.50 EUR (SUCCESS)
--- PRINTER PAPER LOW ---
[DEBUG] BARCODE SCANNER REBOOTED
=== NETWORK SYNC OK ===
[2026-10-15 08:31:10] [POS_02] item=Muffin price=999.00 NOK (REFUND)
"""

sample_records = parse_log(sample_text)

print(f"Records found: {len(sample_records)}  (expected: 2)")
for record in sample_records:
    print(record)
```

```
Records found: 2  (expected: 2)
{'timestamp': '2026-10-15 08:30:00', 'register_id': 'POS_01', 'item':
'Latte', 'price': 4.5, 'currency': 'EUR', 'status': 'SUCCESS'}
{'timestamp': '2026-10-15 08:31:10', 'register_id': 'POS_02', 'item':
'Muffin', 'price': 999.0, 'currency': 'NOK', 'status': 'REFUND'}
```

Task 2: Recursive utility functions

1. Write a recursive directory search function that finds all log files in a nested directory and returns a list of `pathlib.Path` objects:
   - Convert the input path to a `pathlib.Path` object.
   - Iterate through items using `Path.iterdir()`.
   - If an item is a directory, recursively search inside it.

- If an item is a file with a matching extension (`.log`), add its Path object to a list.
- Constraint: Do NOT use `Path.glob()`, `Path.rglob()`, or `os.walk()`. You must implement explicit recursion.

2. Metadata about the different stores can be found in the `data/store_config.json` file. This file contains deeply nested dictionaries and lists (see example below). Write a recursive function that:
   - Traverses any combination of nested dictionaries and lists found in this file.
   - Returns a set (type hint: `set[str]`) of all register id strings where `"active": true`.

Example config.json:

```
{
  "company": "Campus Coffee",
  "regions": [
    {
      "region_name": "Main Campus",
      "stores": [
        {
          "name": "Library Cafe",
          "registers": [
            {"id": "POS_01", "active": true},
            {"id": "POS_02", "active": false}
          ]
        }
      ]
    }
  ]
}
```

2.1.1: Recursive search for log files

```
def find_log_files(directory: str | Path, extension: str = ".log") ->
list[Path]:   # the function returns a list of Path objects
    """
    Find all files with the given extension in the specified
directory.
    """

    directory = Path(directory)
    found: list[Path] = []

    for item in sorted(directory.iterdir()):                # ensures
the same order on every computer
        if item.is_dir():
            found.extend(find_log_files(item, extension))   #
recursively search subdirectories for log files
        elif item.is_file() and item.suffix == extension:
            found.append(item)                              # the file
```

```
has the correct extension, add it to the list

    return found
```

2.1.2: Code verification

```
"""
The data generator creates 3 stores × 3 registers, so we expect 9 log
files.
"""

log_files = find_log_files(DATA_DIR)

print(f"Log files found: {len(log_files)}  (expected: 9)")
for path in log_files:
    print("  ", path)
```

```
Log files found: 9  (expected: 9)
   data\2026\library_cafe\POS_01\register_sales.log
   data\2026\library_cafe\POS_02\register_sales.log
   data\2026\library_cafe\POS_03\register_sales.log
   data\2026\science_building\POS_01\register_sales.log
   data\2026\science_building\POS_02\register_sales.log
   data\2026\science_building\POS_03\register_sales.log
   data\2026\student_union\POS_01\register_sales.log
   data\2026\student_union\POS_02\register_sales.log
   data\2026\student_union\POS_03\register_sales.log
```

2.2.1: Recursive search for active registers

```
def find_active_registers(node: dict | list | object) -> set[str]:  #
the function returns a set of strings
    """
    Recursively search a nested structure of dicts and lists for
register IDs.
    """

    active: set[str] = set()        # the set of active register IDs
found so far

    if isinstance(node, dict):

        # if the node is a dictionary, check if it has an active key
with value True and an "id" key
        if node.get("active") is True and "id" in node:
            active.add(node["id"])

        # recursively search all values in the dictionary
        for value in node.values():
            active |= find_active_registers(value)
```

```
    elif isinstance(node, list):
        for element in node:                                    # search
every list element
            active |= find_active_registers(element)

    return active
```

2.2.2: Code verification

```
"""
In the generated configuration, POS_01, POS_02 and POS_03 are active
and POS_04 is not
"""

with open(DATA_DIR / "store_config.json", encoding="utf-8") as file:
    store_config = json.load(file)

active_registers = find_active_registers(store_config)
print("Active registers:", sorted(active_registers))        #
expected: ['POS_01', 'POS_02', 'POS_03']
```

```
Active registers: ['POS_01', 'POS_02', 'POS_03']
```

Task 3: Functions for cleaning and analysing data using NumPy

1. Write a function for removing price outliers that:
   - Converts input price data to a 1D NumPy float array.
   - Detects hardware glitch outliers (999.00) where a value deviates from the mean by more than 3 standard deviations:
     $$¿ x - \mu \vee ¿ 3\sigma$$
   - Replaces detected outlier values with the median of the valid prices.
   - Returns the cleaned 1D array.
2. Write a function `compute_refund_ratio(sales_count: np.ndarray, refund_count: np.ndarray) -> np.ndarray` that:
   - Computes the element-wise refund ratio using NumPy array operations:
     $$\text{Refund Ratio} = \frac{\text{Refund Count}}{\text{Sales Count} + 10^{-6}}$$
   - The addition of $\epsilon = 10^{-6}$ prevents zero-division errors.

3.1.1 Defining outliers and how to deal with them

```
def remove_price_outliers(price) -> np.ndarray:
    """converts the price data to a numpy float array"""

    price_array = np.array(price, dtype=float).ravel()      # Ravel
turns it to 1D
```

```
    mean, std = price_array.mean(), price_array.std()       # Creates
mu and sigma
    outlier = np.abs(price_array - mean) > 3 * std          # Finds
the outliers


    if outlier.any():
        price_array[outlier] = np.median(price_array[~outlier])  #
Replaces the outliers with the median
    return price_array
```

3.2 Refund ratio

```
def compute_refund_ratio(sales_count: np.ndarray, refund_count:
np.ndarray) -> np.ndarray:
    """calculates the refund ratio"""

    sales_count = np.asarray(sales_count, dtype=float)      # Counting
amount of refunds in the statuses list
    refund_count = np.asarray(refund_count, dtype=float)    # Counting
amount of sales in the statuses list

    refund_ratio = refund_count/(sales_count * 1e-6)    # Calculating
the refund ratio

    return refund_ratio
```

Task 4: Data analysis using Pandas

1. Write a function that:
   - Converts the list of transaction dictionaries read from the log files into a `pandas.DataFrame`.
   - Converts the `timestamp` column to `datetime` and sets it as the DataFrame index.
   - Sorts the DataFrame chronologically by timestamp.
2. Write a function that has the DataFrame you just created as input, groups data by item and resamples transactions in 1-hour windows (1h), calculating total price (sum).
3. Write a function that has the DataFrame as input and creates a pivot table displaying total transaction revenue grouped by item across status categories (`SUCCESS`, `REFUND`, `CANCELLED`).
4. Write a function that has the DataFrame as input and calculates the refund ratio per item by calling the function you made in task 3.

4.1 Turning the log_files into a dataframe

```
def build_dataframe(log_files: list[dict]) -> pd.DataFrame:
    """Turning lists of transaction dicts into a DataFrame indexed
sorted by datetime timestamp."""
```

```
    log_df = pd.DataFrame(log_files)                                    #
Turning the lists into a dataframe
    log_df["timestamp"] = pd.to_datetime(log_df["timestamp"])   #
turning the time into datatime

    return log_df.set_index("timestamp").sort_index()           #
Sorting by data time while returning
```

4.2 Total price of 1 hour windows

```
def hourly_revenue_per_item(log_df: pd.DataFrame) -> pd.Series:
    """Total price per item in 1-hour windows"""

    # creates a new group for each item and it's price per hour
    log_df = log_df.groupby("item")["price"].resample("1h").sum()

    return log_df
```

4.3 Pivot table per item

```
STATUS_ORDER = ["SUCCESS", "REFUND", "CANCELLED"]

def status_pivot_table(log_df: pd.DataFrame) -> pd.DataFrame:
    """Total revenue per item (rows) across status categories
(columns)."""

    pivot = log_df.pivot_table(index="item", columns="status",
values="price",aggfunc="sum", fill_value=0.0)

    pivot.reindex(columns=STATUS_ORDER, fill_value=0.0)

    return pivot
```

4.4 Refund ratio per item

```
def item_refund_ratios(log_df: pd.DataFrame) -> pd.DataFrame:
    """Refund ratio per item"""

    # Counts the status statements for each item
    counts = (log_df.groupby(["item", "status"]).size()
                .unstack(fill_value=0)
                .reindex(columns=STATUS_ORDER, fill_value=0))

    # Calculates the items sale succses ratio
    ratio = compute_refund_ratio(counts["SUCCESS"].to_numpy(),
counts["REFUND"].to_numpy())

    return pd.DataFrame({
        "sales_count": counts["SUCCESS"],
        "refund_count": counts["REFUND"],
```

```
        "refund_ratio": ratio,
    })
```

Task 5: Create full workflow

Combine the functions you have made into a workflow that does the following:

1. Recursively find all log files and extract active register IDs.
2. Ingest text logs and parse records into a list of dictionaries.
3. Build the initial Pandas DataFrame.
4. Apply NumPy outlier cleaning to remove 999.00 price glitches.
5. Apply currency correction: multiply price by 10.0 for all transactions where `currency == "NOK"`.
6. Generate resampled reports, pivot tables and refund ratios.
7. Save all output files listed in the introduction (Scenario: Overview of inputs and outputs for this assignment).

```
def run_pipeline(data_dir: Path = DATA_DIR, output_dir: Path =
OUTPUT_DIR) -> dict:
    output_dir.mkdir(exist_ok=True)

    # 1. discover files
    log_files = find_log_files(data_dir)
    config = json.loads((data_dir /
"store_config.json").read_text(encoding="utf-8"))

    # Finds active registers
    active_ids = find_active_registers(config)

    print(f"Found {len(log_files)} log files; active registers:
{sorted(active_ids)}")


    # 2. ingest text logs + parse records into a list of dictionaries
    records = []


    for file in log_files:
        text, encoding = read_log_file(file)
        file_records = parse_log(text)

        print(f"  {file.relative_to(data_dir)}: {encoding:<10}
{len(file_records)} records")

        records.extend(file_records)


    n_before = len(records)
```

```
    records = [r for r in records if r["register_id"] in active_ids]

    print(f"Parsed {n_before} records, kept {len(records)} from active
registers")


    # 3. building the dataFrame
    df = build_dataframe(records)


    # 4. outlier cleaning
    n_glitches = int((df["price"] == 999.0).sum())
    df["price"] = remove_price_outliers(df["price"].to_numpy())

    print(f"Outlier cleaning: {n_glitches} x 999.00 glitches, max
price now {df['price'].max():.2f}")


    # 5. currency correction (NOK registered at EUR scale)
    nok = df["currency"] == "NOK"
    df.loc[nok, "price"] *= 10
    df["price"] = df["price"].round(2)      # remove floating point
noise (e.g. 76.10000000000001)


    # 6. reports, pivot tables and refunds
    hourly = hourly_revenue_per_item(df)
    pivot = status_pivot_table(df)
    refund_ratio = item_refund_ratios(df)


    # 7. saving all files from the introduction
    (output_dir / "hourly_revenue.json").write_text(
    hourly.to_json(date_format="iso", indent=2),
    encoding="utf-8")

    df.to_csv(output_dir / "cleaned_transactions.csv")
    pivot.to_csv(output_dir / "status_pivot_table.csv")
    refund_ratio.to_csv(output_dir / "item_refund_ratios.csv")

    print("Saved:", (p.name for p in output_dir.iterdir()))

    return {"df": df, "hourly": hourly, "pivot": pivot, "refund
ratio": refund_ratio}
```